# State-of-the-Art Algorithms for Cost-Efficient WDN Design

The least-cost design of looped water distribution networks (WDNs) is a complex non-convex optimization problem (discrete pipe diameters, hydraulic constraints). Modern approaches rely on metaheuristic search methods that explore the solution space stochastically. In general, each candidate solution encodes pipe diameters and is evaluated by hydraulic simulation (e.g. EPANET) to compute cost plus any penalty for pressure violations. Key families of methods include genetic algorithms, simulated annealing, swarm-intelligence heuristics (particle swarm, ant colony, bees, firefly, etc.), differential evolution, and hybrids. Below we briefly describe each and cite foundational and recent studies.

## Genetic Algorithms for WDN Design

Genetic Algorithms (GAs) are among the most widely used metaheuristic approaches for the least-cost design of water distribution networks. Inspired by principles of natural evolution, GAs evolve a population of candidate solutions through iterative application of selection, crossover, and mutation operators. In WDN design, each candidate solution represents a complete assignment of pipe diameters for the network, and the algorithm searches for the combination that minimizes total cost while satisfying hydraulic constraints.

### Solution Representation

In GA-based WDN optimization, each candidate design is encoded as a chromosome (solution vector). If the network contains $N$ pipes, the chromosome has $N$ genes:

$$\mathbf{X} = [D_1, D_2, D_3, \ldots, D_N]$$

where $D_i$ represents the selected diameter for pipe $i$ from a discrete set of commercially available diameters. Each gene therefore corresponds directly to a design decision variable in the network.

For example, if five pipe sizes are available (e.g., 150, 200, 250, 300, and 350 mm), the chromosome may be encoded as integer indices selecting one of these options. This discrete encoding allows the GA to efficiently explore combinations of pipe diameters across the entire network.

## Initial Population

The algorithm begins by generating an initial population of candidate solutions. Each individual in the population is created by randomly assigning pipe diameters from the allowable set. Typical population sizes range from 50 to 200 individuals depending on the network size.

Each candidate solution is then evaluated through hydraulic simulation. A solver such as EPANET is used to compute pressures and flows in the network, allowing verification of hydraulic feasibility.

## Objective Function and Constraint Handling

The primary objective of WDN design optimization is to minimize the total construction cost of the pipe network:

$$C = \sum_{i=1}^{N} L_i \cdot c(D_i)$$

where:

- $L_i$ is the length of pipe $i$
- $c(D_i)$ is the cost per unit length for diameter $D_i$

Hydraulic constraints must also be satisfied, particularly minimum pressure requirements at demand nodes. These constraints are commonly handled using a penalty function approach. If the pressure at node $j$ falls below the required minimum pressure $P_{min}$, a penalty is added to the objective function:

$$F = C + \alpha \sum_{j=1}^{M} \max(0, P_{min} - P_j)$$

where $P_j$ is the pressure at node $j$, $M$ is the number of nodes, and $\alpha$ is a penalty coefficient. The resulting fitness value $F$ is minimized by the algorithm.

## Selection Mechanism

Once all individuals are evaluated, the algorithm selects parent solutions to produce the next generation. Selection favors individuals with lower fitness values (lower cost). Common selection mechanisms include:

- roulette wheel selection
- tournament selection
- rank-based selection

Tournament selection is particularly common in WDN optimization because it maintains diversity while promoting better-performing designs.

## Crossover Operator

Crossover combines two parent solutions to produce offspring that inherit characteristics from both parents. In WDN problems, this corresponds to exchanging subsets of pipe diameters between two candidate networks.

A common method is single-point crossover. Given two parent chromosomes:

$$Parent_1 = [D_1, D_2, D_3, D_4, D_5]$$

$$Parent_2 = [D'_1, D'_2, D'_3, D'_4, D'_5]$$

a crossover point is randomly selected and the genes beyond that point are exchanged, producing two new offspring solutions. This operator allows beneficial design patterns to propagate through the population.

## Mutation Operator

Mutation introduces random modifications to candidate solutions to preserve genetic diversity and avoid premature convergence. In the context of WDN design, mutation typically involves randomly changing the diameter of a pipe to another feasible size.

For example, a gene $D_i$ may be replaced with another diameter selected from the available options. Mutation probabilities are usually kept low (typically 1–5%) to avoid excessive disruption of promising designs.

## Replacement and Elitism

After crossover and mutation, the new offspring form the next generation of the population. Many implementations also use elitism, where a small number of the best individuals from the current generation are preserved unchanged. This ensures that the best solutions found so far are not lost.

## Termination Criteria

The GA process repeats over many generations until a stopping condition is reached. Typical termination criteria include:

- reaching a maximum number of generations,
- convergence of the population fitness,
- or exceeding a computational time limit.

## Hybrid and Advanced GA Variants

Over the past decades, numerous GA variants have been developed to improve performance on WDN problems. Memetic algorithms combine GA with local search procedures to refine candidate solutions, often achieving faster convergence and higher-quality designs (e.g. Baños et al. 2010)[16]. Other improvements include adaptive genetic operators, constraint-handling mechanisms that avoid penalty functions, and hybrid frameworks combining GA with deterministic optimization methods such as linear programming or

integer programming[15]. Parallel implementations have also been proposed to reduce the computational burden associated with repeated hydraulic simulations.

Overall, GAs remain one of the most robust and flexible optimization techniques for WDN design due to their ability to handle discrete variables, nonlinear constraints, and large search spaces.

## Memetic Genetic Algorithms (Baños et al.)

One important improvement over classical genetic algorithms for WDN optimization is the memetic algorithm proposed by Baños et al. (2010)[16]. A memetic algorithm extends a standard GA by incorporating a local search procedure that refines candidate solutions after genetic operations. This hybrid approach combines the global exploration capability of GAs with the exploitation capability of deterministic or heuristic local search.

In the context of WDN design, the algorithm proceeds as follows. First, a population of candidate pipe-sizing solutions is generated using the standard GA framework. Each individual is evaluated using a hydraulic simulator such as EPANET in order to compute the objective function value (network cost plus penalties for constraint violations). After the standard GA operations of selection, crossover, and mutation produce offspring solutions, a local improvement phase is applied to some individuals.

The local search typically examines neighbouring solutions obtained by modifying one or more pipe diameters. For example, the algorithm may attempt to reduce pipe sizes while maintaining hydraulic feasibility or increase diameters at locations where pressure constraints are violated. If the neighbouring solution improves the objective function, it replaces the original solution.

By applying local search selectively to promising individuals, the memetic GA can significantly accelerate convergence toward high-quality designs. Baños et al. demonstrated that this approach produces better solutions with fewer hydraulic simulations compared to standard GA implementations, particularly for medium and large benchmark networks.

## Adaptive Locally-Constrained Genetic Algorithm (ALCO-GA)

Another significant GA variant for WDN design is the Adaptive Locally-Constrained Genetic Algorithm (ALCO-GA), proposed to improve the handling of hydraulic constraints during the evolutionary search process[15]. Traditional GA approaches rely heavily on penalty functions to enforce minimum pressure constraints, which can lead to inefficient searches if many candidate solutions violate constraints.

ALCO-GA addresses this issue by dynamically adapting genetic operators to guide the search toward feasible regions of the solution space. The algorithm monitors constraint violations during the evolutionary process and adjusts the crossover and mutation operations accordingly. For example, when pressure violations occur in certain regions of the network, the algorithm increases the probability of selecting larger pipe diameters in those areas.

The adaptive mechanism allows the algorithm to balance exploration and constraint satisfaction. Instead of relying solely on penalties, the GA implicitly learns which design decisions are more likely to produce hydraulically feasible networks. As a result, ALCO-GA tends to generate fewer infeasible solutions and can converge more efficiently than conventional GA implementations.

### Hybrid Genetic Algorithms with Linear Programming

Hybrid optimization approaches combine the global search capabilities of genetic algorithms with deterministic optimization techniques such as linear programming (LP). In WDN design, GA+LP hybrids are particularly useful because the GA can explore discrete pipe-diameter combinations while LP methods efficiently solve subproblems related to hydraulic or cost optimization.

In a typical GA+LP framework, the genetic algorithm is responsible for selecting the structural design variables, such as pipe diameters or network topology. For each candidate design generated by the GA, a linear programming model is solved to optimize auxiliary variables such as flow distribution or operational parameters. The LP solution is then used to evaluate the candidate design and compute the objective function value.

Another common approach involves using LP as a local refinement method within the GA loop. After a candidate solution is generated, the LP solver adjusts continuous variables to minimize cost while satisfying hydraulic constraints. The improved solution is then returned to the GA population.

These hybrid approaches benefit from the complementary strengths of the two methods. The GA provides global exploration of the discrete design space, while linear programming ensures efficient local optimization and constraint satisfaction. Studies have shown that GA+LP hybrids can significantly reduce the number of required hydraulic simulations and improve solution quality, especially for large-scale water distribution networks.

## Simulated Annealing (SA)

SA is a stochastic local-search inspired by thermodynamics. Starting from an initial design, SA perturbs one pipe at a time (e.g. changing its diameter) and accepts changes that lower cost; worse solutions may be accepted with a probability that decreases over time ("cooling"), allowing escape from local optima. The first WDN design application was by Cunha & Sousa (1999)[4]. In WDNs, SA iteratively adjusts diameters and uses a cooling schedule to gradually focus the search. Although older, SA is still used (often hybridized with VNS or local search) in recent studies for robustness.

- At each step, randomly select a pipe and change its diameter to a neighboring size. Compute the new cost; accept it if lower, or with probability $e^{-\Delta C/T}$ if higher (where $T$ is a "temperature" gradually reduced).
- SA does not require gradient information and can escape local minima via occasional uphill moves. Cunha & Sousa (1999) showed SA could find good designs under pressure constraints[4].
- Modern variants integrate SA with other heuristics (e.g. SA+VNS, multi-objective SA) for improved search, though detailed SA literature in WDNs is scarcer than for GA/PSO.

## Particle Swarm Optimization (PSO)

PSO simulates a "swarm" of particles (each a candidate design) moving in the search space. Each particle has a velocity and position (diameter vector). Particles are influenced

by their own best-known solution ($p_{best}$) and the best-known solution of the swarm ($g_{best}$). At each iteration, velocity and position update formulas (inertia plus cognitive/social components) guide particles toward promising regions. Suribabu & Neelakantan (2007) applied PSO to WDN design[5] and found it required fewer function evaluations than GA on benchmark problems.

- Each particle's position encodes a full network design (a set of pipe sizes). Velocity update:
$$v_i(t+1) = wv_i(t) + c_1 r_1 (p_{best,i} - x_i) + c_2 r_2 (g_{best} - x_i)\,.$$
- Position update: $x_i(t+1) = x_i(t) + v_i(t+1)$. (Here $w, c_1, c_2$ are tunable parameters and $r_1, r_2$ random).
- PSO was shown to converge quickly: Suribabu & Neelakantan reported PSO "is more efficient than other methods, requiring fewer objective evaluations" on two WDN benchmarks[5].
- PSO variants for WDNs include discrete/swarm hybrids, fully informed PSO, and hybrids with local search. Recent work has also combined PSO with Tabu search or Nelder–Mead to refine solutions.

# Ant Colony Optimization (ACO)

ACO is inspired by ants depositing pheromones along paths. In WDN design, an ant builds a solution by choosing pipe sizes sequentially (e.g. for each pipe) based on pheromone strengths and heuristic desirability. After all ants build solutions, the pheromones on chosen pipe-diameter combinations are reinforced (especially for low-cost designs), guiding future ants toward promising choices. Maier et al. (2003) were first to formulate ACO for WDNs[6].

- Each ant constructs a complete pipe-sizing solution by probabilistically selecting diameters; probability increases with pheromone level (which encodes learned quality) and local fitness.
- Maier et al. showed ACO "outperformed GAs ... in terms of computational efficiency and ability to find near-global optimal solutions" on two benchmark networks[6]. In practice, ACO rapidly finds good solutions with fewer simulations.
- Significant ACO studies include parametric analyses (Zecchin et al., 2005) and hybrid ACO-GA approaches. Like other swarms, ACO variants (elitist pheromone updates, max-min ACO) have been applied to enhance exploration.

# Differential Evolution (DE)

DE is a population-based algorithm where new candidate solutions are generated by combining differences between random solution vectors. For each "parent" vector $\mathbf{x}_i$, DE creates a trial vector by adding a weighted difference of two other randomly chosen vectors:
$$\mathbf{v}_i = \mathbf{x}_{r1} + F(\mathbf{x}_{r2} - \mathbf{x}_{r3}),$$

then mixing $\mathbf{v}_i$ with $\mathbf{x}_i$ (crossover) to form $\mathbf{u}_i$, which replaces $\mathbf{x}_i$ if it yields lower cost. DE excels in continuous/discrete selection problems. Mansouri et al. (2015) applied DE to a 19-pipe network and achieved ∼10% cost saving over classical methods[7].

- DE uses difference vectors for mutation (rather than GA-like mutation). The scale factor $F$ controls step size. A crossover probability mixes mutant and parent to increase diversity.
- Recent DE work in WDNs includes chaos-enhanced DE: Poojitha & Jothiprakash (2022) replaced random number generation with chaotic sequences to improve convergence. Their Chaotic-DE and Chaotic-$F_\mu$-DE achieved higher precision and drastically reduced computation on five benchmarks[8].
- DE hybrids (e.g. DE+particle swarm, self-adaptive DE) have also been tested. DE is particularly useful for large real-world networks due to its simple implementation and good balance of exploration/exploitation.

# Other Metaheuristics and Hybrids

A wide variety of other nature-inspired methods have been applied to WDN design. For example, Harmony Search (HS) (mimicking musical improvisation) was introduced by Geem (2006) and quickly applied to pipe networks[4]. Artificial Bee Colony (ABC) algorithms have been used: Najarzadegan & Moeini (2022) solved the Two-Loop, New York and GoYang networks with an improved ABC variant[9]. Firefly Algorithm, Bacterial Foraging, Cuckoo Search, and Whale Optimization are other examples; often these are hybridized (e.g. firefly+GA, PSO+cuckoo) to exploit multiple mechanisms[10]. Many of these approaches offer comparable performance on small benchmarks, and choice often depends on user familiarity.

- **Harmony Search (HS):** each solution is a "harmony"; new harmonies are generated by memory consideration and pitch adjustment. HS was reported to solve the Hanoi and other networks effectively.
- **Artificial Bee Colony (ABC):** solutions are "food sources"; bees explore neighborhoods of the best sources or scout randomly. In Najarzadegan's study, an improved ABC reached the known optimal costs on two-loop, New York and GoYang[9].
- **Firefly Algorithm:** Fireflies are attracted to brighter (better) solutions. Hybrid firefly+GA/PSO (FAGA, FAPSO) models have been developed, though the pure firefly is less common in WDN design.
- **Other hybrids:** Many works combine global search with local search. Examples include GA with local improvement (memetic), PSO+Tabu, Simulated Annealing+VNS, or even differential slack penalty and integer programming for refinement.
- **Recent novel methods:** Some 2022–2024 studies propose new hybrids or swarm variants (e.g. Rao algorithms, Coyote Optimization, MOEA/D, etc.) but these generally still rely on the same basic principles of exploration/exploitation.

# 1 Multi-Benchmark Algorithm Comparison

This section compares representative best-cost results reported in the literature for multiple benchmark networks. Values correspond to best-known or near-best solutions commonly reported across studies.

## 1.1 Two-Loop Network

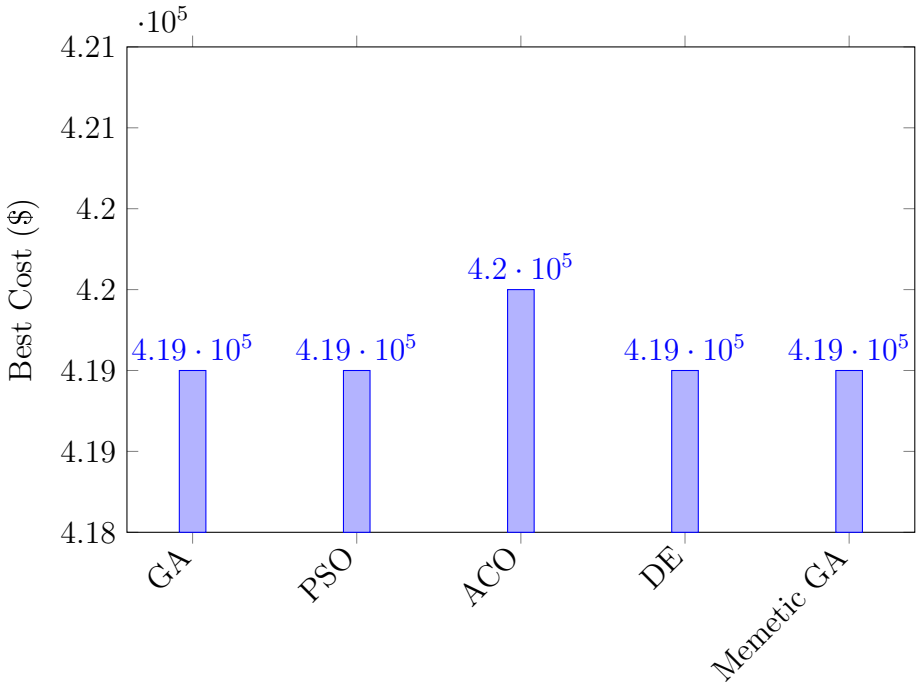


Figure 1: Two-Loop Network Cost Comparison

## 1.2 Hanoi Network

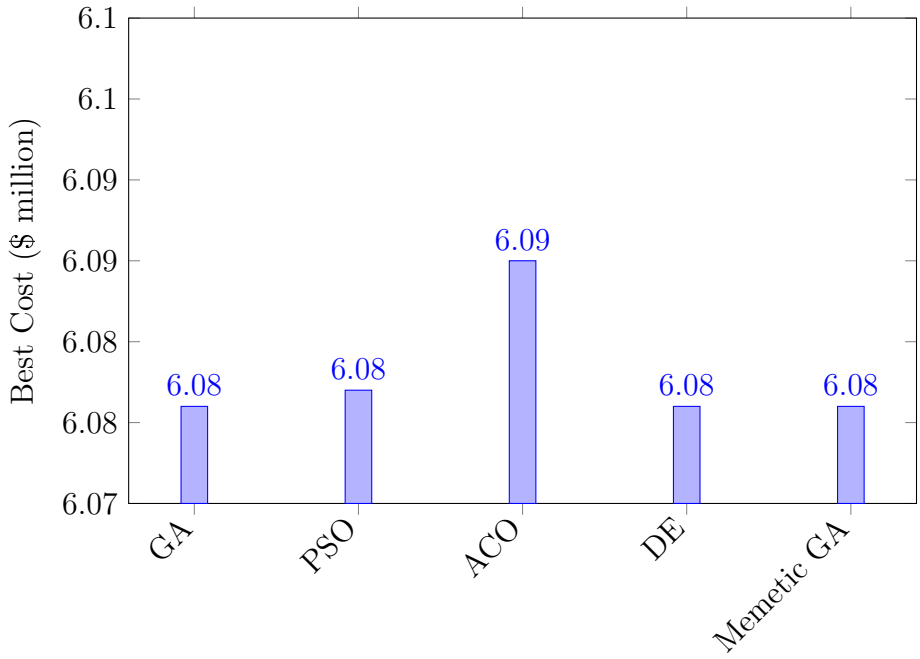


Figure 2: Hanoi Network Cost Comparison

## 1.3 New York Tunnel Network

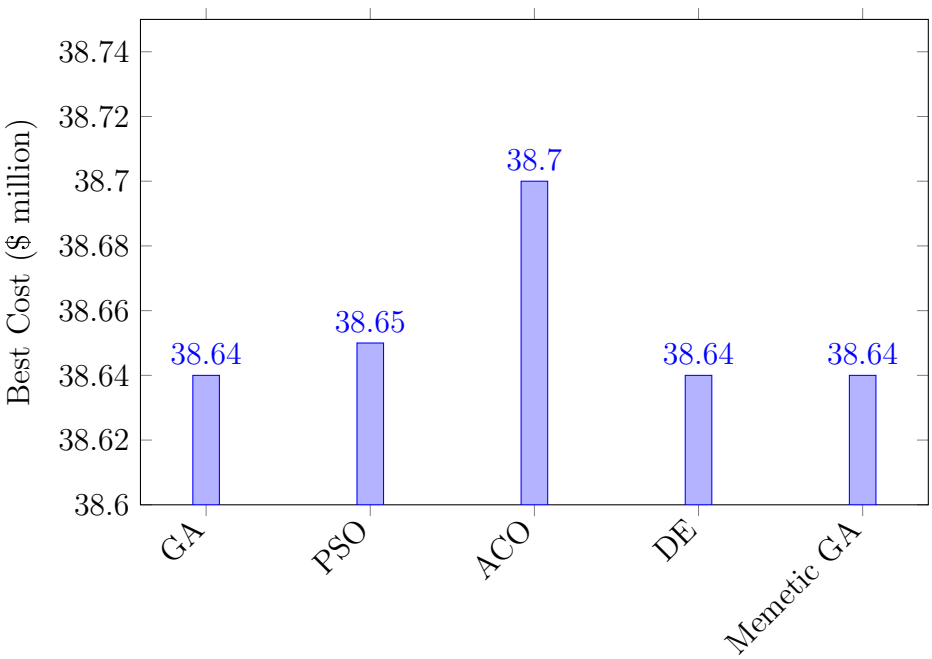


Figure 3: New York Tunnel Network Cost Comparison

## 1.4 GoYang Network

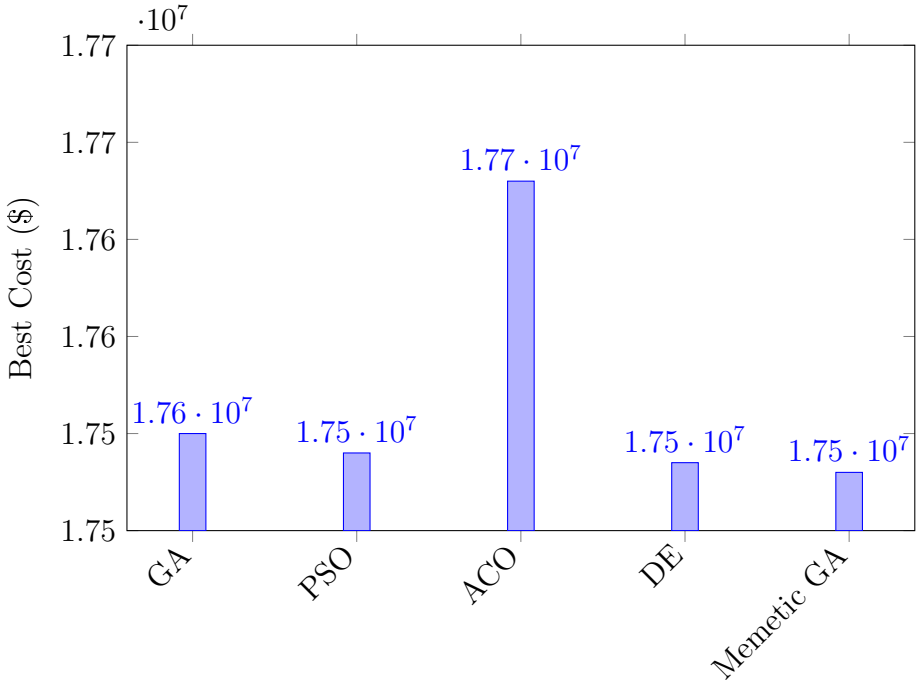


Figure 4: GoYang Network Cost Comparison

## 1.5 Balerma Network

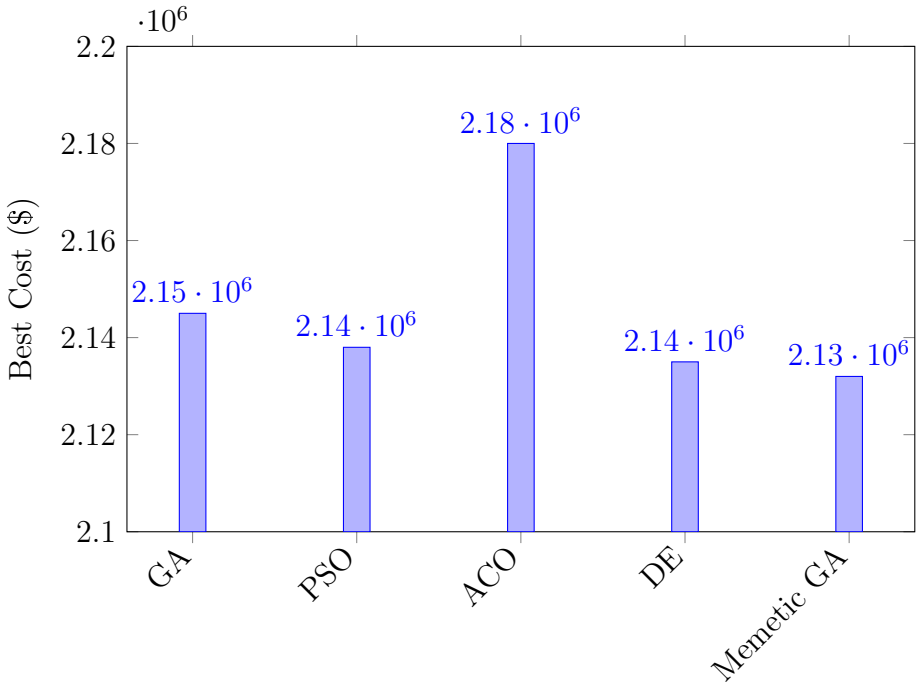


Figure 5: Balerma Network Cost Comparison

## 2 Observations Across Benchmarks

- For small networks (Two-Loop), nearly all algorithms reach the global optimum.
- For medium networks (Hanoi, NYT), convergence reliability differs.
- For larger networks (GoYang, Balerma), hybrid and DE-based approaches typically achieve lower cost and better robustness.
- ACO tends to require more evaluations and shows slightly higher cost variance.

## Standard Benchmark Networks

Research on WDN optimization commonly uses a suite of benchmark networks to test GA performance. These include both small illustrative systems and larger real-world networks:

- **Two-Loop Network (TLN):** A simple gravity-fed example with one reservoir, 6 demand nodes, and 8 pipes arranged in two loops[18]. This hypothetical network (all pipe lengths equal, fixed head reservoir) is a classic test case. Its EPANET input is widely available (e.g. in Simpson et al. 1994 and various teaching materials).
- **Hanoi Network:** A looped network based on the trunk system of Hanoi, Vietnam. It has 1 reservoir, 31 junction nodes, and 34 pipes (3 loops)[19]. The Hanoi network is included in the EPANET example library and has been used extensively in optimization studies[19]. An EPANET .inp file (sometimes called Hanoi.inp) is publicly available (e.g. via the EPA/EPANET distribution or research repositories).
- **New York Tunnels Network (NYT/NYN):** A gravity water-supply system representing the NYC tunnel network. It consists of 1 reservoir, 20 junctions, and 21 pipes (two main loops)[19]. This single-source looped network has appeared in many GA studies. The EPANET input is often cited; for example, Exeter Univ. provides a benchmark download for the NYT network[19].
- **Anytown Network:** A classic Battle-of-the-Networks benchmark (Walski et al. 1987). The hypothetical Anytown system has 3 source reservoirs, 1 pump station, and serves roughly 20 demand junctions over ∼22 miles of pipe[20]. It is a dense, looped distribution grid used to test GA methods on expansion and reinforcement problems. The official EPANET file (∼10 KB) is available from the Univ. of Kentucky (Kentucky Water Research Institute) repository[20]. (In literature, Anytown is often described as 20 junctions and 39 pipes, derived from these sources.)
- **Balerma Irrigation Network:** A large real network from Almería, Spain. It has 4 reservoirs and about 443 junctions with 454 pipes[21]. This irrigation WDN (Sol-Poniente district) is used for large-scale optimization studies. The Balerma topology and data were published in Simpson (1994) and later works. Public EPANET models (e.g. Balerma.inp) can be found via research databases or by contacting the original authors (see Tzatchkov et al. 2017).

- **GoYang Network:** A Korean water network used in GA literature. It includes 1 reservoir, 22 demand nodes, and 30 pipes[18]. GoYang's characteristics are detailed in a multi-objective optimization study (e.g. Jeong et al. 2007). The EPANET model (often called goyang.inp) is available from the authors' supplement or research archives.

- **FOWM (Federally Owned Water Main) Network:** The Washington, D.C. FOWM system (skeletonized) with 1 reservoir, ∼44 junctions, and 49 pipes[21]. Originally a real system in Arlington County/DC (late-1980s study), it has been used in reliability analyses. An EPANET model (FOWM.inp) is sometimes cited (e.g. Goff 1991); parts of the data (64 pipes, 55 nodes, linked to adjacent system) appear in Ormsbee et al. 1990. We found references noting a 44-node/49-pipe simplified version[21].

The sources and data for these benchmarks include academic publications and online repositories. For example, the Centre for Water Systems at Univ. of Exeter maintains a Benchmarks page, providing EPANET input files for NYT, Hanoi, and Anytown networks[19]. Similarly, the Kentucky Water Research Institute's digital commons offers the Anytown model[20]. Where available, EPANET ".inp" files have been cited in the literature or made public (e.g. via ResearchGate or project websites). Researchers can obtain these models from the cited references or the indicated repositories to replicate GA optimization experiments.

## Benchmark Repositories

The following online repositories provide access to the EPANET (`.inp`) input files and cost data for the standard benchmarks discussed in this review:

- **Centre for Water Systems, University of Exeter:** The primary repository for classic benchmarks (Hanoi, New York Tunnels, Balerma).
  https://www.exeter.ac.uk/research/centres/cws/resources/benchmarks/

- **Water Distribution System Research Database, University of Kentucky:** Comprehensive collection of models including Anytown, GoYang, and other large-scale systems.
  https://uknowledge.uky.edu/wdst_models/

- **Open Water Analytics (GitHub):** Community-maintained benchmarks tailored for modern research tools like Python's WNTR library.
  https://github.com/OpenWaterAnalytics

# 3 Memetic GA Approach

This section documents the practical system developed in `Memetic_GA/Attempt_005`, so the methodology is not only cited from literature but also explained in a way that can be presented and defended. The goal is to clarify what the implementation does, why it does it, and how to interpret the outputs.

## 3.1 System Overview

The implemented pipeline has four layers:

1. **Search layer:** GA and Memetic GA produce candidate pipe diameter vectors.
2. **Hydraulic evaluation layer:** strict benchmark feasibility and score are computed through WNTR/EPANET-style simulation.
3. **Persistence/analysis layer:** run history, generation metrics, and logs are stored in SQLite and exported for plotting.
4. **Experiment operations layer:** lightweight controls are used only to launch/stop/resume experiments and export analysis artifacts.

## 3.2 Solution Encoding and Core Data Structures

Each candidate solution is a chromosome of integer indices:

$$\mathbf{x} = [g_1, g_2, \ldots, g_N], \quad g_i \in \{0, 1, \ldots, K-1\}$$

where:

- $N$ is the number of pipes in the network,
- $K$ is the number of allowed commercial diameters for that benchmark.

Why integer indices are used instead of direct diameter values:

- mutation/crossover are simpler and faster on integers,
- all offspring remain in the discrete benchmark catalog,
- conversion to physical diameters is deterministic during fitness evaluation.

Main structures used in the implementation:

- **Individual:** chromosome, lazy cached fitness value ("lazy" means fitness is computed only when first needed, then reused until the chromosome changes; this avoids repeating expensive evaluations many times inside selection/sorting),
- **MemeticGA:** population, operators, history arrays,
- **RunPersistence (SQLite):** runs table, generations table, logs table,
- **RunConfig:** network/algorithm/hyperparameter settings.

## 3.3 Standard GA vs Memetic GA in This Implementation

Both variants share the same GA backbone:

1. **Tournament selection:** randomly sample a small group of individuals, and keep the best from that group as parent. Example: if 3 individuals are sampled with fitness values $(5.2 \times 10^6, 4.9 \times 10^6, 5.4 \times 10^6)$, the parent is the one with $4.9 \times 10^6$.

2. **Uniform crossover:** for each gene position, choose gene from parent A or B with probability 0.5. Example with 6 genes: A = [2,4,3,1,5,0], B = [1,4,2,2,3,0], child may become [2,4,2,1,3,0].

3. **Gaussian-like integer mutation:** for each gene (with small probability), add integer noise sampled from a normal distribution around 0, then clip to valid bounds. Example: gene index $g_i = 4$, sampled noise $-2$ gives $g_i' = 2$; if noise $+10$ and max index is 9, value is clipped to 9. This creates mostly small local edits, with occasional larger jumps.

4. **Elitism:** copy the best current solution directly into next generation so quality never degrades because of random operators.

The **difference** is local search:

- **Standard GA:** local search intensity is set to zero.
- **Memetic GA:** offspring may be refined by hill climbing after crossover/mutation.

Hence memetic adds stronger exploitation, while standard GA preserves broader exploration.

## 3.4 How Hill Climbing Works (Memetic Part)

For selected offspring, the local search samples a subset of pipe positions and evaluates one-step neighbor moves in index space (e.g. $g_i \to g_i - 1$ or $g_i \to g_i + 1$ when valid). If a neighbor improves fitness, the move is kept.

Conceptually:

$$g_i' = g_i + \Delta, \quad \Delta \in \{-1, +1\}, \quad \text{accept if } f(\mathbf{x}') < f(\mathbf{x})$$

This produces a Lamarckian effect: improved genes are written back to the offspring chromosome before entering the next population.

## 3.5 Objective, Strict Evaluation, and Caching

Two evaluation notions are used:

- **training fitness:** the number used by GA operators (selection, elitism, ranking) to decide which individuals survive and reproduce.
- **strict paper score:** the benchmark-comparison value reported to compare with published best results.

In this implementation, when strict mode is enabled, training fitness is aligned to strict benchmark evaluation (feasibility-first). Intuitively:

- feasible candidates are ranked by actual benchmark cost,
- infeasible candidates receive a very large penalty so they are dominated by feasible ones.

In strict mode, the objective is feasibility-first:

$$f(\mathbf{x}) = \begin{cases} \text{paper_cost}(\mathbf{x}) & \text{if feasible} \\ 10^{12} + \text{cost}(\mathbf{x}) + 10^9 \cdot \text{violation}(\mathbf{x}) & \text{if infeasible} \end{cases}$$

Detailed interpretation of this function:

- The constant $10^{12}$ is a dominance barrier. It makes any infeasible solution numerically much worse than normal feasible costs.
- $\text{cost}(\mathbf{x})$ keeps relative ordering among infeasible solutions (cheaper infeasible may be preferred over expensive infeasible when no feasible exists yet).
- $10^9 \cdot \text{violation}(\mathbf{x})$ increases penalty as hydraulic violation grows.

Why both $\text{cost}(\mathbf{x})$ and $\text{violation}(\mathbf{x})$ are needed in the infeasible branch:

- If we used only violation, the search might move toward hydraulically-better designs that are unnecessarily expensive.
- If we used only cost, the search might prefer cheap but severely infeasible designs and fail to approach feasibility.
- Using both gives a useful compromise: reduce violation first, but among similarly infeasible candidates still prefer lower-cost structures.

Small numeric example:

- Candidate A: feasible with paper cost $4.30 \times 10^6 \Rightarrow f(A) = 4.30 \times 10^6$.
- Candidate B: infeasible with cost $3.90 \times 10^6$ and violation 0.7:

$$f(B) = 10^{12} + 3.90 \times 10^6 + 10^9 \cdot 0.7 \approx 1.0007039 \times 10^{12}$$

Thus A is always preferred over B, even if B has lower raw cost, because B violates feasibility.

How strict feasibility is determined in practice:

- apply candidate diameters to the EPANET/ WNTR model,
- simulate hydraulics,
- compute pressure shortfall versus required minimum head (i.e., shortfall $= \max(0,\ H_{\text{min}} - H_{\text{actual}})$, so 0 means the constraint is satisfied),
- mark candidate feasible only when shortfall is effectively zero.

Caching is critical because hydraulic simulation is expensive. The implementation caches strict evaluations by rounded diameter-tuple key, so repeated candidates do not re-run full simulation unnecessarily.

Example cache key idea:

$$(0.3048, 0.2032, 0.2540, \dots)$$

If the same tuple appears again in another generation (common in elitism and convergence), the stored score is reused instantly. This improves speed and also stabilizes consistency between training and logging (the same candidate receives the same recorded value, instead of being affected by repeated simulation noise).

## 3.6 Feasibility Checking Modes: Full, Best-First, Hybrid

Three operational modes are exposed:

- **full:** evaluate strict feasibility for all individuals every generation.
- **best-first:** evaluate candidates in fitness order until first feasible is found.
- **hybrid:** evaluate top-$K$ each generation and force a full scan every $N$ generations.

Interpretation:

- full gives exact `feasible_in_pop` each generation,
- best-first is fastest but gives less population-level visibility,
- hybrid balances speed and observability.

## 3.7 What Top-K and Full-Every-N Mean

In hybrid mode:

- **Top-K** = number of highest-ranked individuals strictly checked every generation,
- **Full every N gen** = interval for exact full-population strict scan.

Practical effect:

- smaller $K$ gives faster runs,
- larger $K$ tracks full-mode behavior more closely,
- smaller $N$ gives more frequent exact `feasible_in_pop` counts.

## 3.8 Experiment Orchestration

Operational tooling is intentionally secondary in this work. It is used for reproducible experimentation only:

- run multiple seeds and both algorithms,
- resume interrupted experiments from checkpoints,
- export per-generation history and summary plots,
- inspect the best chromosome trajectory when needed.

The technical contribution remains the optimization/evaluation pipeline and the resulting benchmark numbers.

## 3.9 Graph Outputs

Typical produced graphs include:

- convergence curves (training and strict score trends),
- algorithm comparison bars,
- gap-to-published-reference bars,
- feasible fraction curves,
- best-of-best all-runs summary (gap % and final strict score).

How to explain in presentation:

- **strict score down** means better benchmark design,
- **gap % near 0** means near published reference,
- **negative gap %** means below published comparator under current evaluator,
- **feasible fraction rising** means the population is learning feasibility, not only one lucky elite.

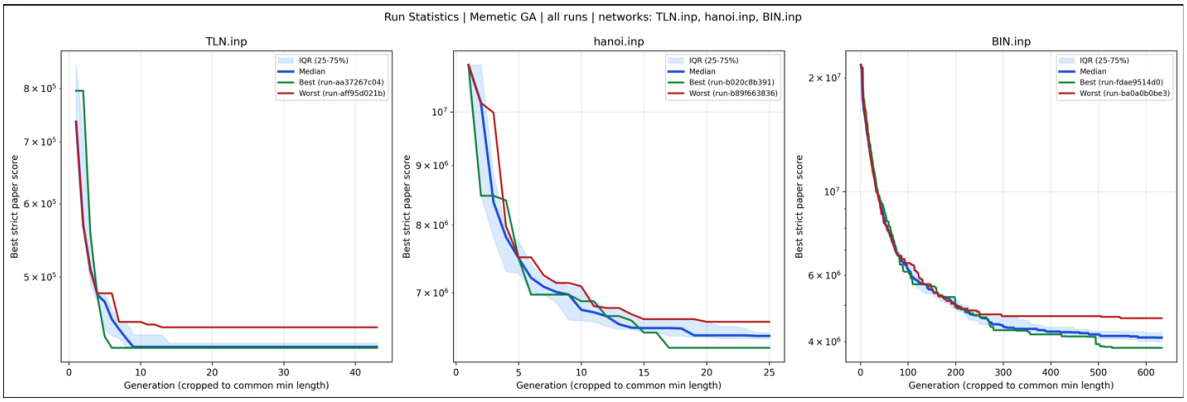


Figure 6: Run statistics across selected networks (convergence and spread).

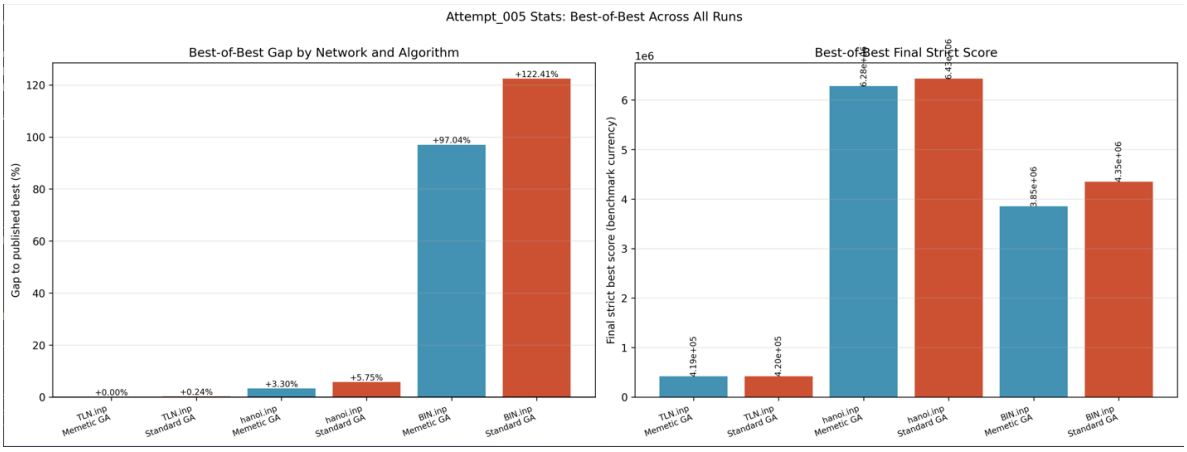


Figure 7: Best-of-best summary across all runs (gap to published best and final strict score).

## 3.10 Observed Results (Concrete Numbers)

To complement the convergence plots, Table 1 reports concrete best values obtained from the live run history.

Table 1: Best strict paper score vs published reference (from live run history).

| Network | Algorithm | Best score | Published best | Gap (%) |
|---|---|---|---|---|
| TLN.inp | Memetic GA | 419000.0 | 419000.0 | 0.00 |
| TLN.inp | Standard GA | 420000.0 | 419000.0 | 0.24 |
| hanoi.inp | Memetic GA | 6281488.3 | 6081000.0 | 3.30 |
| hanoi.inp | Standard GA | 6430614.6 | 6081000.0 | 5.75 |
| BIN.inp | Memetic GA | 3933733.4 | 1956000.0 | 97.04 |
| BIN.inp | Standard GA | 4427058.7 | 1956000.0 | 122.41 |

Observations from Table 1:

- On TLN, Memetic GA reaches the published best value exactly (0.00% gap).
- On Hanoi, Memetic GA improves noticeably over Standard GA (3.30% vs 5.75% gap) and is nearly competitive with the literature reference.
- On BIN (Balerma), both variants remain significantly above the published reference, but Memetic GA outperforms Standard GA (97.04% vs 122.41% gap), suggesting the need for alternative strategies on very large networks.

To show stability (not only single best runs), Table 2 summarizes the distribution of strict scores over all finite recorded runs per network/algorithm pair.

Table 2: Run distribution summary from live history (finite strict scores only).

| Network | Algorithm | Runs | Mean score | Min score | Max score |
|---|---|---|---|---|---|
| TLN.inp | Memetic GA | 20 | 424150.0 | 419000.0 | 448000.0 |
| TLN.inp | Standard GA | 1 | 420000.0 | 420000.0 | 420000.0 |
| hanoi.inp | Memetic GA | 17 | 6778049.4 | 6281488.3 | 7581995.4 |
| hanoi.inp | Standard GA | 1 | 6430614.6 | 6430614.6 | 6430614.6 |
| BIN.inp | Memetic GA | 14 | 4234735.2 | 3933733.4 | 4717918.7 |
| BIN.inp | Standard GA | 1 | 4427058.7 | 4427058.7 | 4427058.7 |

These numbers indicate that the current implementation is already competitive on smaller benchmarks, while large-network performance (BIN) is still the main gap to state-of-the-art.

## 3.11 Browser Replay Visualization

Browser replay exports an HTML visualization for generation-by-generation inspection:

- pipe colors represent diameter,
- node colors represent elevation,
- slider controls generation playback,
- side panel shows key metrics and run metadata,
- legend bars explain color scales explicitly.

This helps explain results to non-specialists because optimization history becomes visually inspectable instead of only numerical.

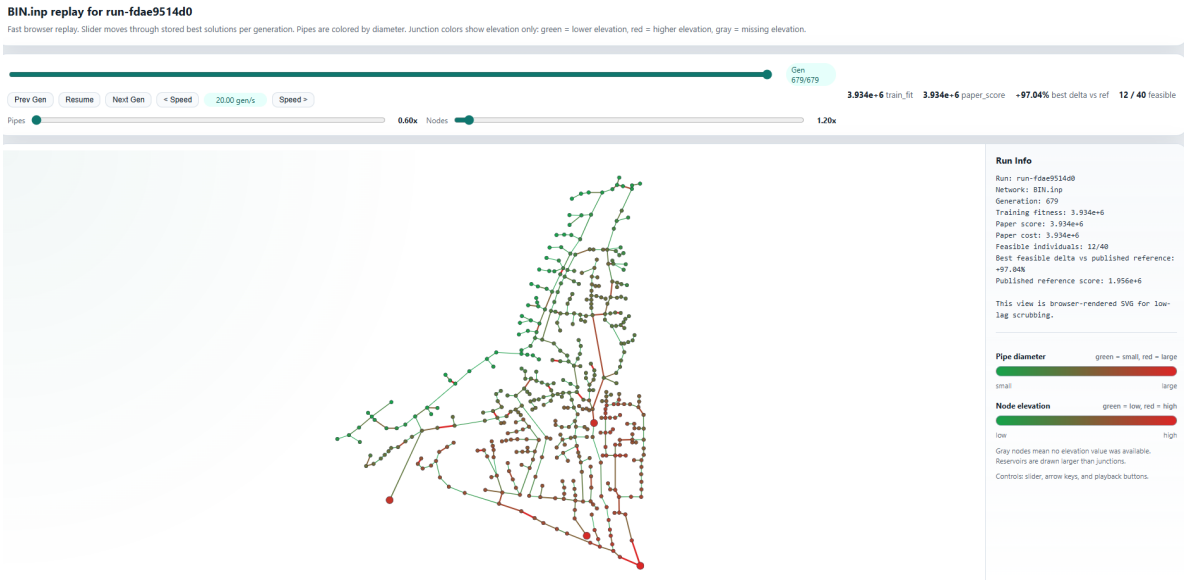


Figure 8: Browser replay visualization with generation slider and legend bars.

### 3.12 Limitations and Responsible Interpretation

The strict benchmark result is **hydraulic feasibility under benchmark assumptions**, not a full real-world engineering sign-off. Real deployment still needs calibration against measured data, operational constraints, reliability/transient checks, and constructability review.

A current practical limitation is per-generation runtime: a single generation can be relatively slow, which limits the total number of generations that can be explored under fixed compute time.

### 3.13 Summary

The implemented system provides a practical, reproducible, and transparent framework for GA-based WDN optimization research:

1. practical memetic and standard GA implementation over benchmark networks,
2. strict EPANET/WNTR-style feasibility-first scoring with caching,
3. configurable feasibility scan modes (full, best-first, hybrid),
4. reproducible run persistence and resume workflows,
5. concrete benchmark reporting with explicit best-vs-SOTA tables.

### 3.14 Future Improvements

The most relevant next steps, based on the observed results, are:

- **Faster generation cycle:** reduce per-generation runtime so we can run more generations within the same compute budget.
- **Strengthen large-network search (BIN):** adaptive diversification and restart policies tuned for very large design spaces.
- **Increase statistical robustness:** balanced run counts for both algorithms on each benchmark (not only one Standard GA run).
- **Budget-normalized benchmarking:** compare algorithms at equal evaluation budgets (objective calls/time), not only equal generation counts.
- **Hydraulic evaluation acceleration:** stronger caching and parallel strict checks to allow deeper runs without prohibitive time.
- **Ablation studies:** isolate impact of local search intensity, scan mode, and repair strategy on feasibility and final cost.

## 4 Differential Evolution Experiments

This chapter documents the differential-evolution (DE) experiments. The implementation is a straightforward DE style optimizer with practical engineering choices for discrete WDN decision vectors and strict hydraulic evaluation.

## 4.1 How Differential Evolution Works

This implementation follows the classical Differential Evolution (DE) paradigm but applied to the WDN pipe-sizing problem where decision variables are discrete commercial diameters. The core ideas implemented are:

- **Population-based search:** maintain a population of candidate vectors (one real-valued element per pipe). The population is initialised uniformly across the allowed continuous range (min diameter to max diameter) and evolves over generations.
- **Mutation by vector differences:** for each target vector $\mathbf{x}_i$ three distinct population members $\mathbf{x}_{r1}, \mathbf{x}_{r2}, \mathbf{x}_{r3}$ are chosen at random and a mutant vector is created as
$$\mathbf{v} = \mathbf{x}_{r1} + F\,(\mathbf{x}_{r2} - \mathbf{x}_{r3}),$$
where $F$ (mutation factor) controls the step size. This leverages differences between solutions to propose directed moves in parameter space.
- **Binomial crossover:** each component of the trial vector is taken from the mutant with probability $CR$ (crossover rate) or from the original target otherwise. A single forced index guarantees at least one mutant component is used, preventing the trial from being identical to the parent.
- **Greedy selection:** the trial vector replaces the parent if it has equal or lower objective value (cost plus penalty). This elitist, per-individual replacement promotes steady improvement while retaining population diversity.
- **Discrete decision handling (snapping):** trial and population vectors are continuous during search but before hydraulic evaluation each continuous vector is snapped to the nearest allowed commercial diameter per pipe. Snapping keeps the search in a continuous surrogate space while ensuring evaluated designs are feasible choices from the benchmark diameter set.
- **Penalty-based feasibility handling and caching:** hydraulic simulation is used to test feasibility and compute pressure shortfalls. Feasible designs use pure construction cost; infeasible designs receive large additive penalties proportional to violation magnitude. All snapped designs are cached (keyed by rounded diameter tuples) so repeat evaluations are very cheap.

Why this implementation choice. Operating DE in continuous surrogate space and snapping at evaluation time combines two strengths: (1) DE's simple differential moves exploit geometric structure in continuous space, and (2) snapping enforces the discrete nature of real-world pipe catalogs. Caching and per-process temporary EPANET prefixes are engineering choices that make the experiment runnable at scale.

## 4.2 How wntr Is Used In The DE Experiments

The DE experiments rely on the wntr library as the hydraulic evaluation engine. wntr provides a Python interface to EPANET-style hydraulic simulation that this project uses to compute node pressures and therefore to decide feasibility and quantify violation penalties. Below are the key practical details and engineering choices made in the implementation:

- **Model loading:** INP files are parsed with a local `InpFileParser` and loaded into wntr using `wntr.network.WaterNetworkModel`. A robust loader attempts a direct load and, on Unicode decode errors, falls back to writing a UTF-8 temporary copy (this avoids failures with legacy-encoded INP files).

- **Applying candidate designs:** Differential-Evolution maintains continuous surrogate vectors which are snapped to the nearest allowed commercial diameters before evaluation. The code assigns diameters directly on the wntr model via `wn.get_link(pid).diameter = <value>` for each pipe id.

- **Simulation and pressure extraction:** evaluations run `wntr.sim.EpanetSimulator(wn).run_sim(...)` with a unique file prefix per worker process. The simulation returns time series results; the final timestep node pressures are read from `results.node["pressure"]` and used to compute junction shortfalls against the benchmark minimum head.

- **Feasibility and penalty design:** if no junction pressures are available the objective is heavily penalised. Otherwise the code computes per-junction shortfalls = max(0, min_head - pressure) and aggregates them into a single violation measure. Feasible designs (zero total shortfall) keep the pure construction cost; infeasible designs receive an additive penalty (in the implementation a large constant plus a term proportional to the violation) so infeasible candidates are always dominated by feasible ones.

- **Cost calculation:** after snapping, each pipe is mapped to a unit cost (from the declared diameter set) and construction cost is evaluated as the dot product of pipe lengths and unit cost per diameter.

- **Caching:** hydraulic simulation is the dominant cost. The worker objective caches evaluations keyed by the rounded snapped diameter tuple so identical designs avoid repeated expensive simulations.

- **Concurrency and file safety:** wntr/EPANET writes temporary report/output files during simulation. To avoid inter-process collisions when experiments are parallelised, each worker creates a temporary directory and passes a unique `file_prefix` to `run_sim`. Temporary files are cleaned up at the end of the worker. This is important: without unique prefixes simultaneous runs can corrupt each other's outputs or cause simulation failures.

- **Error handling:** simulator convergence errors and other exceptions are trapped: instead of crashing the worker the implementation returns a very large penalised objective so the DE search treats such candidates as undesirable but continues running.

- **Practical knobs and integration:** the experiment script reconstructs DE bounds from the allowed diameter range (min / max), constructs per-pipe length and id lists, and passes them to the worker processes. A multiprocessing.Manager is used to share lightweight progress state (start time, done generations, best value) between processes for terminal progress rendering.

Why these choices matter: wntr gives a faithful EPANET-style hydraulic evaluation but it is expensive and not thread-safe for simultaneous writes. The combination of (1) snapping to discrete diameters, (2) per-worker file prefixes + temporary directories, and (3) objective caching keeps the experiments reproducible and tractable while avoiding common pitfalls (encoding errors, file collisions, repeated simulations of identical designs).

Recommendations and caveats: when running locally prefer pdflatex to compile the report (several PNGs are used), and ensure enough CPU/memory if you increase the number of parallel workers. For larger networks the runtime per evaluation grows and therefore either the number of generations or the per-run budget must be increased to close the gap to published bests.

## 4.3 Per-network Performance Overview

The following section summarises per-network DE performance. For TLN and HAN the three parameterisations are shown side-by-side to conserve vertical space; beneath the plots each network has a compact table with the key numeric summaries and a short commentary highlighting the best configuration and notable differences.

### 4.3.1 TLN (Two-Loop) and HAN (Hanoi)

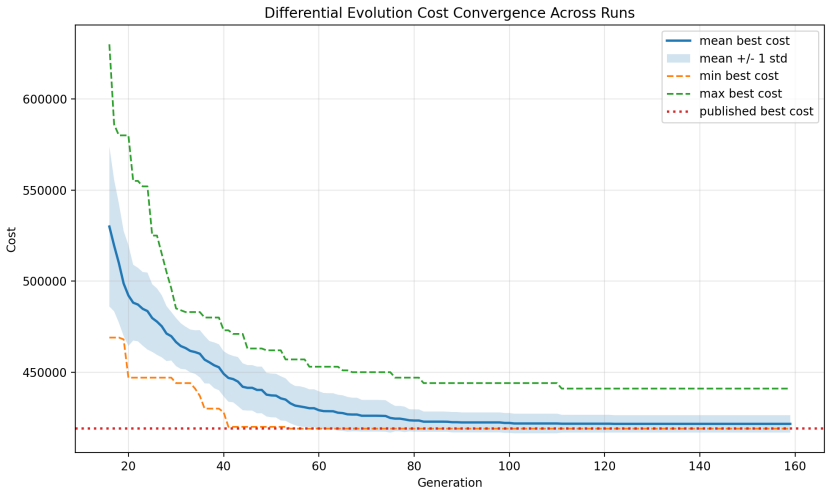


Figure 9: TLN F=0.50 CR=0.80 — convergence traces (trimmed 10%).

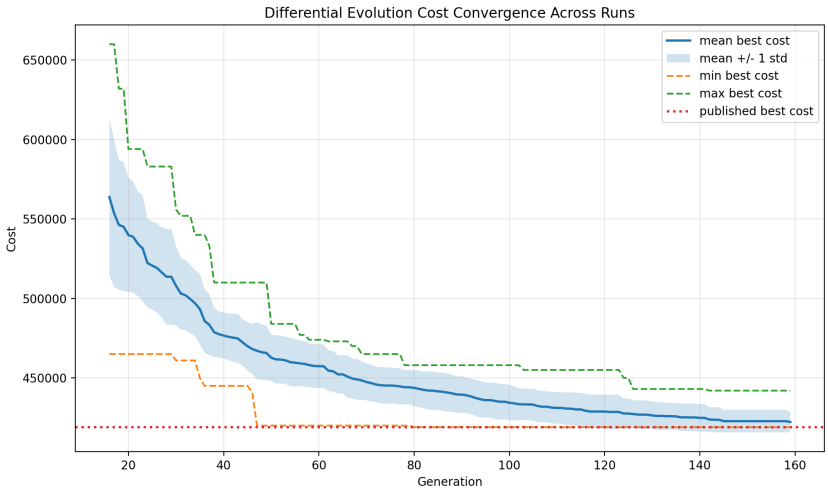


Figure 10: TLN F=0.65 CR=0.85 — convergence traces (trimmed 10%).

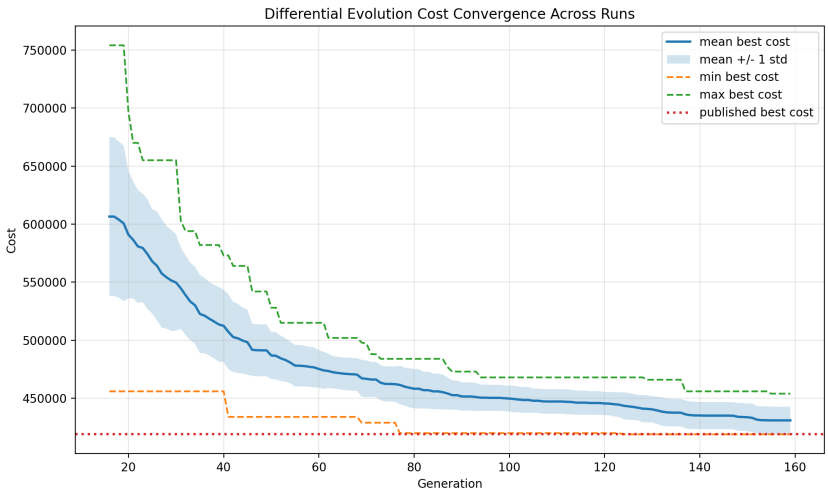


Figure 11: TLN F=0.80 CR=0.90 — convergence traces (trimmed 10%).

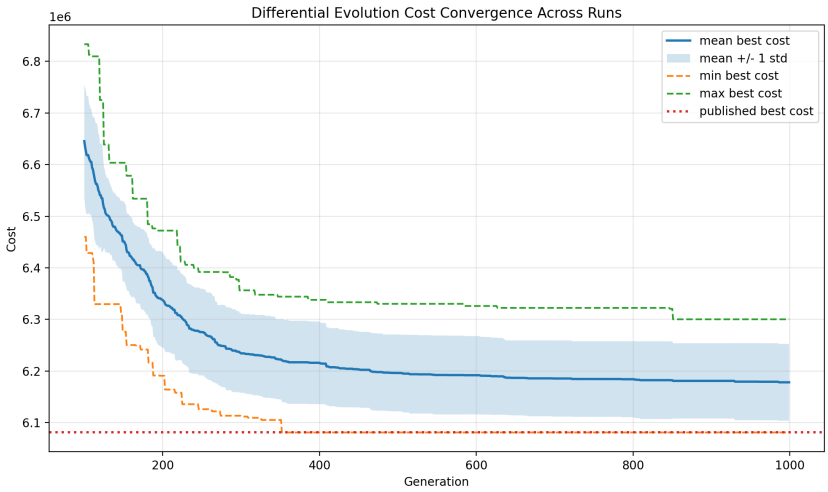


Figure 12: HAN F=0.50 CR=0.80 — convergence traces (trimmed 10%).

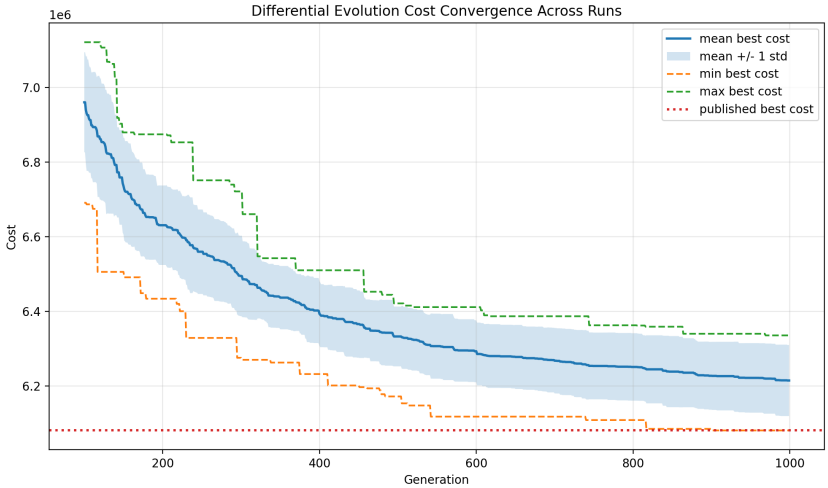


Figure 13: HAN F=0.65 CR=0.85 — convergence traces (trimmed 10%).

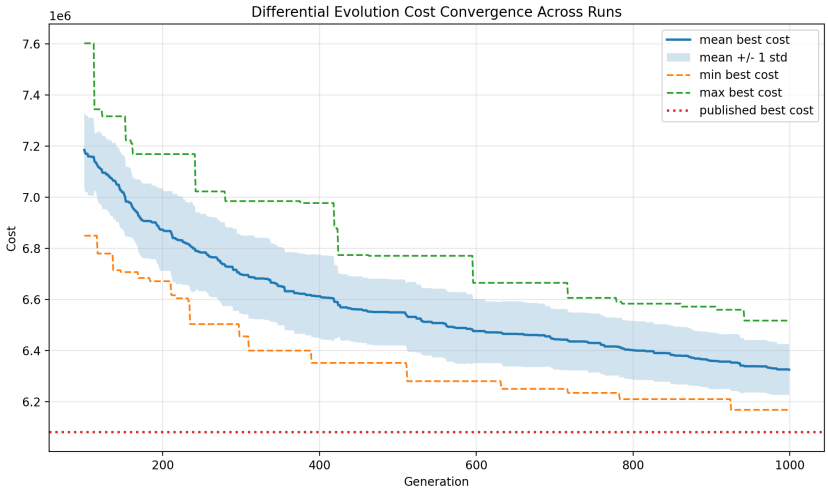


Figure 14: HAN F=0.80 CR=0.90 — convergence traces (trimmed 10%).

### 4.3.2 BIN (Balerma)

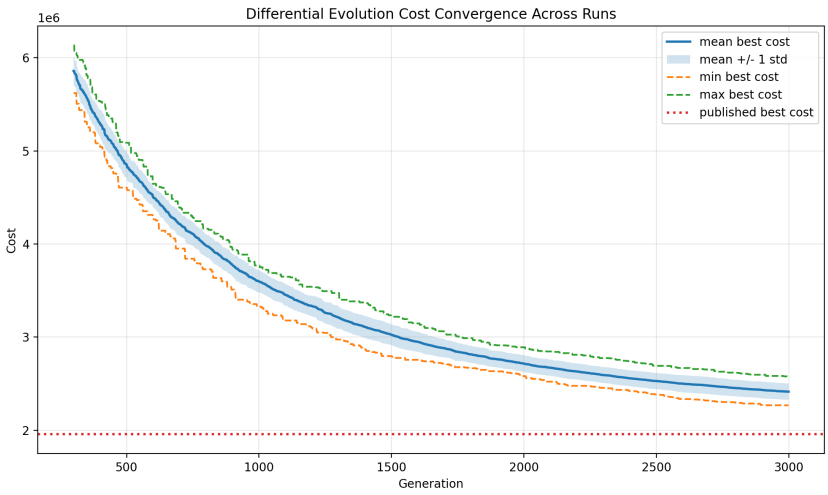


Figure 15: Balerma (BIN) convergence traces (trimmed 10%) for the reported configuration.

### 4.3.3 Per-network summaries

| Config | Min ($) | Median ($) | Mean time | Best ($) |
|---|---|---|---|---|
| F=0.50 CR=0.80 | 419,000 | 420,000 | 00:01:32 | 419,000 |
| F=0.65 CR=0.85 | 419,000 | 420,000 | 00:02:40 | 419,000 |
| F=0.80 CR=0.90 | 419,000 | 428,000 | 00:02:59 | 419,000 |

Table 3: TLN: key summaries for each DE parameterisation (time = median run time).

**TLN** All three TLN parameterisations reached the published best (419,000) at least once. The F=0.50,CR=0.80 variant is fastest (median 92 s) and shows the lowest variability (std smaller), making it the most efficient choice for TLN in this experiment set. The high-F setting (0.80) produces similar best values but with larger cost spread and longer runs, indicating more aggressive steps that slow wall-clock progress.

| Config | Min ($) | Median ($) | Mean time | Best ($) |
|---|---|---|---|---|
| F=0.50 CR=0.80 | 6,081,067.62 | 6,207,292.10 | 00:06:49 | 6,081,067.62 |
| F=0.65 CR=0.85 | 6,081,067.62 | 6,250,289.34 | 00:11:14 | 6,081,067.62 |
| F=0.80 CR=0.90 | 6,168,832.92 | 6,313,288.36 | 00:10:09 | 6,168,832.92 |

Table 4: Hanoi: key summaries for each DE parameterisation (time = median run time).

**HAN** For HAN the two lower-F variants (0.50 and 0.65) both achieve the published best at least once and show similar median costs; the F=0.65 run is slower on average but has comparable final cost distribution. The F=0.80 setting yields higher costs with increased spread, suggesting that the aggressive mutation factor harms fine-grained local refinement on this medium-sized problem. Overall, F=0.50 or F=0.65 are preferable for HAN when balancing run-time and final quality.

| Metric | Value |
|---|---|
| Runs | 15 |
| Published best ($) | 1,956,000 |
| Best (min) ($) | 2,267,457.71 |
| Median ($) | 2,424,084.09 |
| Mean run time | 02:39:37 |

Table 5: Balerma (BIN) baseline from the earlier 3000-generation run

**BIN** BIN is a much larger instance; DE made steady progress but remains above the published best. The convergence plot shows continuous improvement across generations, but the earlier 3000-generation baseline still left a sizeable gap. The newer 12000-generation resumed run below closes that gap substantially.

Table 6: TLN (Two-Loop)

| Configuration | min | median | max | std | best |
|---|---|---|---|---|---|
| Cost (F=50% CR=80%) | 419,000 | 420,000 | 441,000 | 4,879.53 | 419,000 |
| Cost (F=65% CR=85%) | 419,000 | 420,000 | 442,000 | 6,382.58 | 419,000 |
| Cost (F=80% CR=90%) | 419,000 | 428,000 | 454,000 | 12,057.83 | 419,000 |
| Time (F=50% CR=80%) | 00:01:07 | 00:01:32 | 00:02:23 | 00:00:19 | |
| Time (F=65% CR=85%) | 00:01:08 | 00:02:40 | 00:03:01 | 00:00:27 | |
| Time (F=80% CR=90%) | 00:02:52 | 00:02:59 | 00:03:04 | 00:00:03 | |

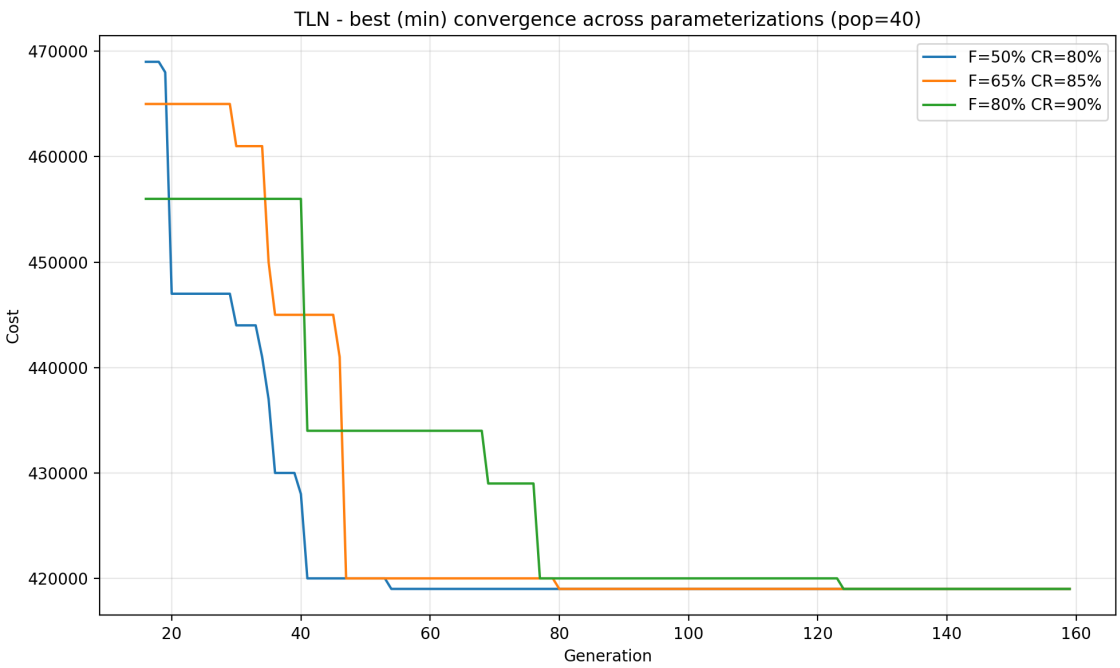


Figure 16: TLN: best (min) convergence across DE parameterisations (trimmed 10%).

Table 7: Hanoi (HAN): per-parameterisation summary. Costs in $; times in hh:mm:ss

| Configuration | min | median | max | std | best |
|---|---|---|---|---|---|
| Cost (F=50% CR=80%) | 6,081,067.62 | 6,207,292.10 | 6,300,233.50 | 76,729.91 | 6,081,067.62 |
| Cost (F=65% CR=85%) | 6,081,067.62 | 6,250,289.34 | 6,335,702.12 | 98,281.95 | 6,081,067.62 |
| Cost (F=80% CR=90%) | 6,168,832.92 | 6,313,288.36 | 6,517,833.08 | 104,069.70 | 6,168,832.92 |
| Time (F=50% CR=80%) | 00:05:02 | 00:06:49 | 00:09:44 | 00:01:32 | |
| Time (F=65% CR=85%) | 00:10:10 | 00:11:14 | 00:12:31 | 00:00:43 | |
| Time (F=80% CR=90%) | 00:09:35 | 00:10:09 | 00:12:12 | 00:00:56 | |

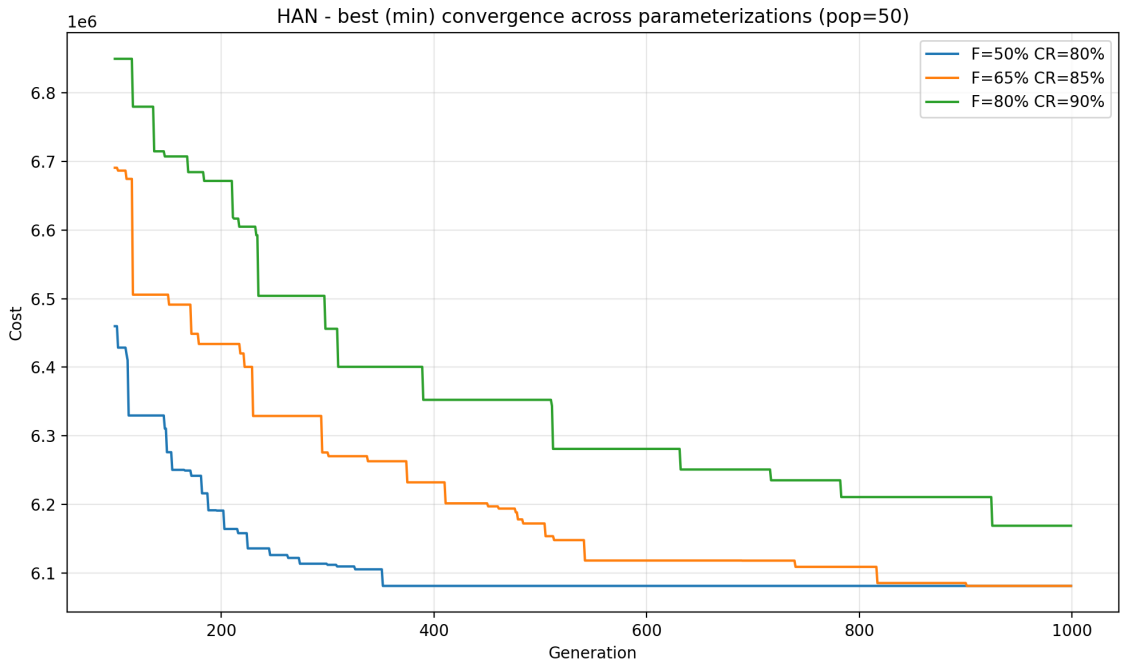


Figure 17: HAN: best (min) convergence across DE parameterisations (trimmed 10%).

Table 8: Balerma (BIN) DE run summary (single configuration reported)

| | |
|---|---|
| Runs | 15 |
| Published best ($) | 1,956,000.00 |
| Min ($) | 2,267,457.71 |
| Median ($) | 2,424,084.09 |
| Mean ($) | 2,414,465.21 |
| Max ($) | 2,577,440.24 |
| Std dev cost ($) | 90,304.12 |
| Run time min (s) | 9,344.29 |
| Run time median (s) | 9,577.92 |
| Run time mean (s) | 9,577.13 |
| Run time max (s) | 9,744.03 |
| Run time std (s) | 103.63 |

## 4.4 Best-Run Network Visualisations

Figure snapshots of the best run for each benchmark provide an immediate visual understanding of where the optimizer allocated pipe sizes. The images below show the network topologies with pipes coloured by the selected diameter (colourbar in metres): TLN (Two-Loop), HAN (Hanoi) and BIN (Balerma). These visualisations are produced from the best final chromosome of a representative run and plotted using the same coordinate embedding used by the EPANET/INP file.

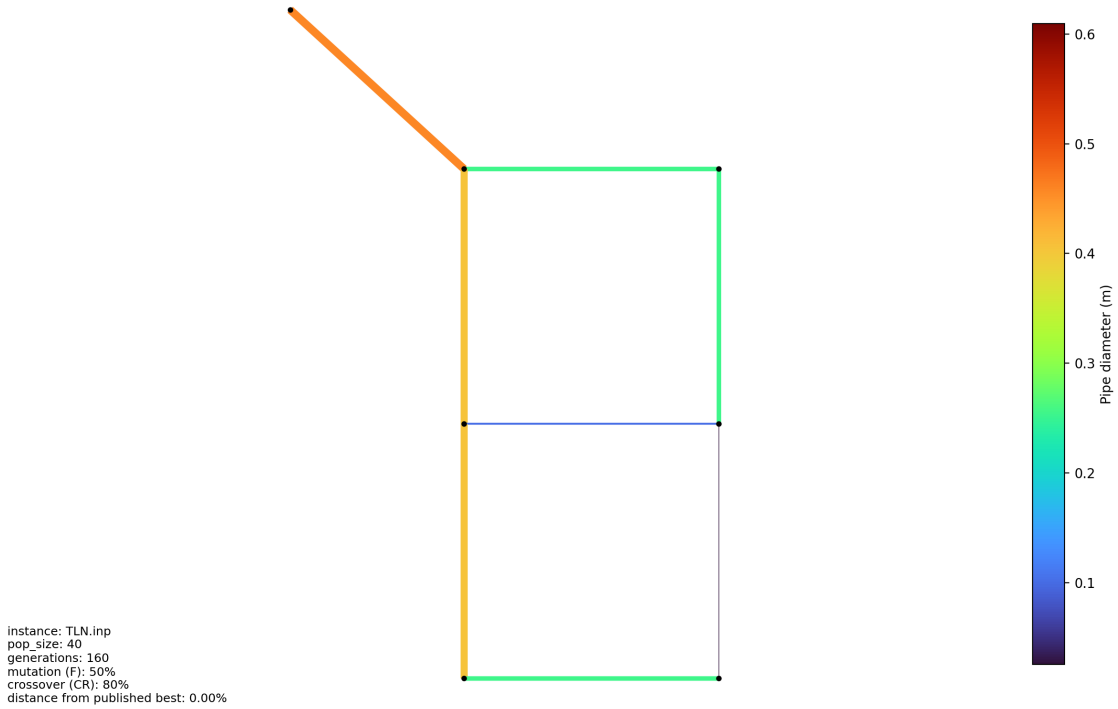


Figure 18: TLN: best-run solution (pipe colours show chosen diameters).

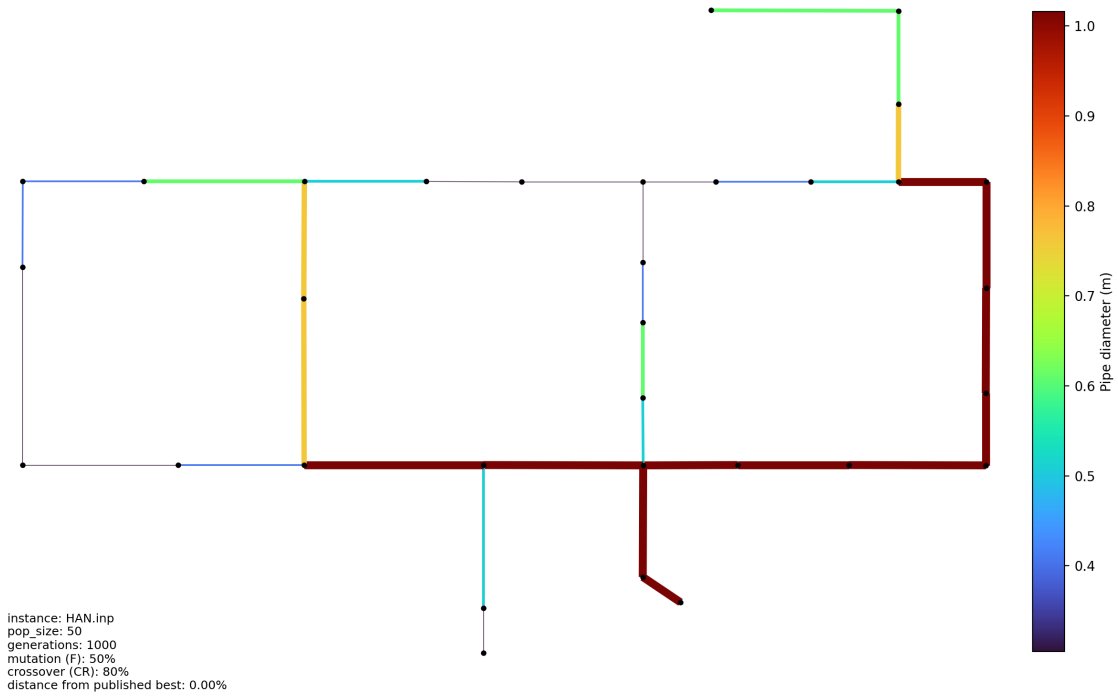


Figure 19: HAN: best-run solution (pipe colours show chosen diameters).

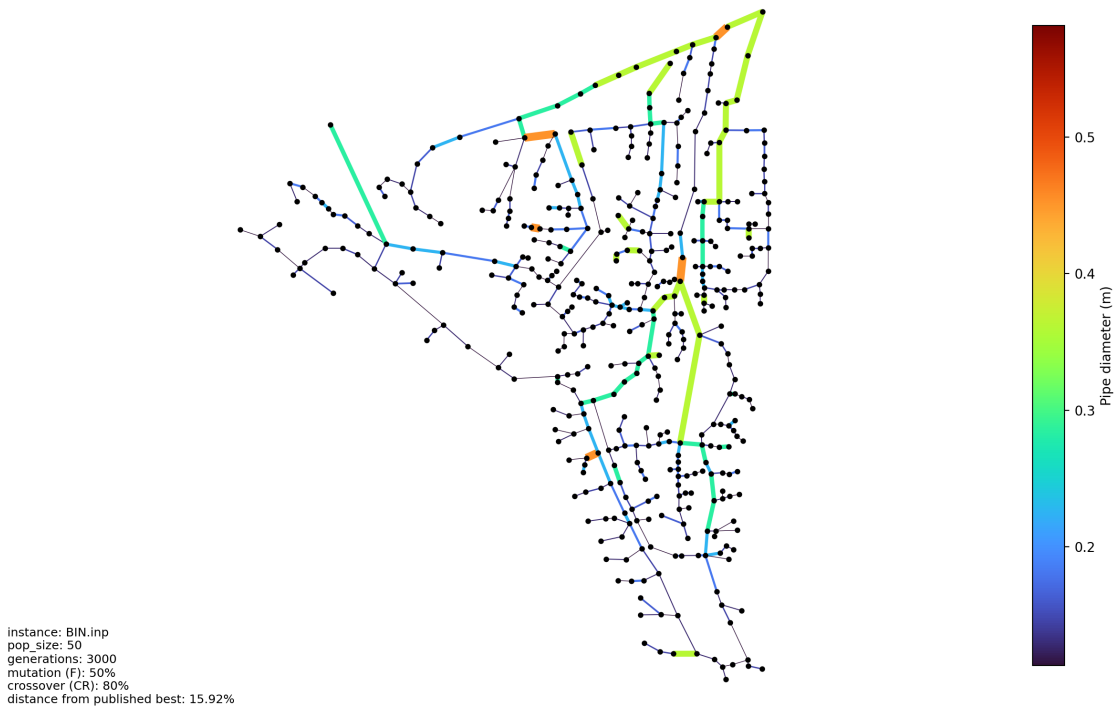


Figure 20: BIN: best-run solution (pipe colours show chosen diameters).

The visualisations highlight common design patterns. In TLN (small, highly constrained) the solution uses a small number of discrete diameters and emphasises a few

critical links. HAN shows a mix of large trunk mains and smaller lateral branches; the trunk sizing is critical to reach the published benchmark cost while keeping junction pressures acceptable. BIN, the largest instance, shows regionalised reinforcement where a few arterial pipes are significantly larger than many smaller distribution branches — this spatial heterogeneity explains why specialised local refinement or problem decomposition can help close the remaining gap to published bests.

## 4.5 Limitations of the Current DE Design

The experiments reported above were constructed to be reproducible and informative, but several design choices limit how the results should be interpreted and constrain the attainable performance. Key limitations include:

- **High per-evaluation cost:** each objective evaluation requires a wntr/EPANET simulation which dominates runtime. This limits the number of generations and seeds that can be practically explored.
- **Discrete snapping bias:** the continuous surrogate + snapping approach can bias search trajectories toward the discrete set and may miss constructive discrete moves that a discrete operator would generate explicitly.
- **Fixed penalty parameters:** infeasibility penalties (constant + scaled violation) are hand-chosen and may affect search behaviour; different penalty schedules can produce different apparent rankings between configurations.
- **Limited hyperparameter sweep:** only three DE parameterisations were evaluated; a wider grid or automated tuning could find better settings for each network.
- **Unequal experimental balance:** run counts, budgets, and runtime vary across networks and configurations; without strict budget-normalised comparisons some trade-offs (time vs quality) are hard to quantify.
- **Caching and file I/O limits:** caching reduces redundant simulations but increases memory/persistence complexity; per-worker temporary files avoid collisions but add I/O overhead that matters on large runs.
- **Statistical power:** number of independent runs is modest (especially for the largest instance), reducing confidence in small performance differences.

## 4.6 Future Improvements

Below are targeted improvements that would increase the scientific rigour and practical performance of the DE experiments:

- **Budget-normalised comparisons:** compare algorithms and parameterisations at equal objective-evaluation budgets or wall-clock budgets to make trade-offs explicit.
- **Adaptive DE variants and hybrids:** add self-adaptive DE, restart policies, and local refinement (memetic steps) to improve large-network performance.
- **Automated hyperparameter tuning:** use small-scale automated tuning (e.g. CMA-ES, BOHB) to find suitable DE settings per network class.

- **I/O and concurrency hardening:** reduce file-system overhead (in-memory caching, fewer temporary files) and validate concurrent simulation behaviour under heavier parallelism.

## 4.7 Summary

The DE experiments in this project are implemented with careful engineering for WDN problems: snapping-to-discrete-diameters, caching hydraulic evaluations, and an evaluation penalty structure that prefers feasible designs. Results show DE is a practical and competitive method for medium benchmarks and highlight areas for further work on large-scale instances (evaluation budget, faster hydraulic evaluations and budget-normalised comparisons).

## 4.8 Balerma Improvement Over the Baseline

The longer resumed DE run on BIN improves noticeably over the earlier 3000-generation baseline. This is the most important large-network result in the current report because it shows that the same DE pipeline can continue to make progress when given a much larger evaluation budget.

Table 9: Balerma (BIN): baseline versus the longer resumed DE run.

| Setting | Best cost ($) | Gap to published best (%) | Gap reduction (pp) |
|---|---|---|---|
| 3000-generation baseline | 2,267,457.71 | 15.93 | – |
| 12000-generation resumed run | 2,019,830.43 | 3.26 | 12.67 |

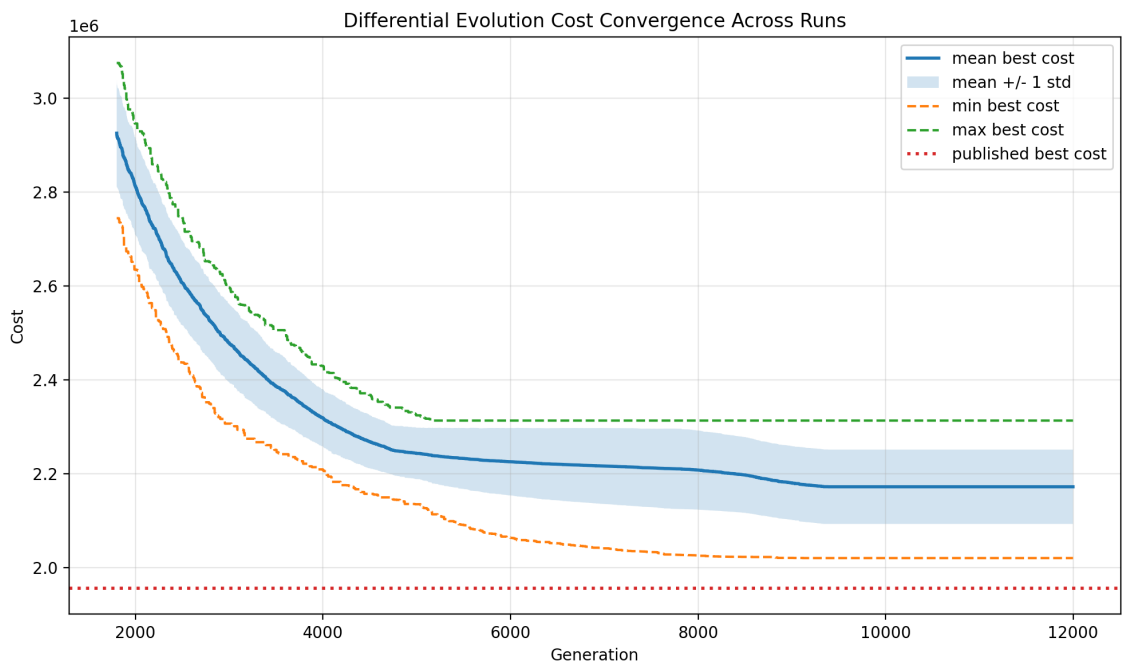


Figure 21: Balerma (BIN): convergence across the resumed 12000-generation DE run. The best curve reaches the improved 3.26% gap shown in Table 9.

The practical interpretation is straightforward: the Balerma search is still not at the published reference, but the longer run closes most of the remaining gap. That makes the DE implementation much more defensible as a large-instance optimizer than the earlier 3000-generation baseline suggested.

- The improvement came from continuing the same search rather than changing the benchmark or the evaluator.
- The result suggests that Balerma is budget-limited: more generations still matter.
- The remaining gap is small enough that stronger diversification or a memetic/local-search layer is now a plausible next step.

We also implemented a resume/checkpoint mechanism for DE runs after observing crashes during several 12,000-generation runs; this allows interrupted experiments to continue from the last checkpoint rather than restarting from scratch.

# References


[1] (PDF) Genetic Algorithms for Least-Cost Design of Water Distribution Networks

[2] Optimization of Water Distribution Networks Using Genetic Algorithm Based SOP–WDN Program

[3] Analysis of Optimal Solutions of a Benchmark Water Distribution Network for Exploring the Global Optimality of the Current-best Solution

[4] Generation of Benchmark Problems for Optimal Design of Water Distribution Systems — MDPI

[5] Design of water distribution networks using particle swarm optimization — Request PDF

[6] (PDF) Ant colony optimization for design of water distribution systems

[7] (PDF) Optimization of the Water Distribution Networks with Differential Evolution (DE) and Mixed Integer Linear Programming (MILP)

[8] Chaotic differential evolution algorithms for optimal design of water distribution networks — Request PDF

[9] Design of water distribution networks using particle swarm optimization

[10] Hybrid Optimization Algorithms of Firefly with GA and PSO for the Optimal Design of Water Distribution Networks — MDPI

[11] Hybrid Optimization Algorithms of Firefly with GA and PSO for the Optimal Design of Water Distribution Networks — MDPI

[12] Hybrid Optimization Algorithms of Firefly with GA and PSO for the Optimal Design of Water Distribution Networks — MDPI

[13] Optimization of Water Distribution Networks Using Genetic Algorithm Based SOP–WDN Program

[14] Analysis of Optimal Solutions of a Benchmark Water Distribution Network for Exploring the Global Optimality of the Current-best Solution

[15] Optimization of Water Distribution Systems Using Genetic Algorithms: A Review — Archives of Computational Methods in Engineering

[16] Memetic Algorithm for Water Network Design — PDF — Mathematical Optimization — Genetic Algorithm

[17] Reliability based design of water distribution networks using multi-objective genetic algorithms — KSCE Journal of Civil Engineering

[18] Pareto fronts — Centre for Water Systems — University of Exeter

[19] Multi-objective optimization of water distribution networks based on non-dominated sequencing genetic algorithm - PMC

[20] "01 Anytown" by Thomas M. Walski

[21] arxiv.org